Higher-Order Functions in Haskell

Basic Combinators

Combinator	CL	Haskell	Type
Identity			
Constant			
Exchange			
Composition			
Application			

Higher-order Functions for Lists and Tuples

Name	Type	Description
`map`		
`filter`		
`zipWith`		
`dropWhile`		
`takeWhile`		
`curry`		
`uncurry`		
`zip`		
`span`		

One Recursive Function to Rule Them All

```
foldr ::

foldr f b []     = b
foldr f b (x:xs) = f x (foldr f b xs)
```

Challenge Problem: Define the functions `map`, `filter`, and `takeWhile` in one line using `foldr`. Your definitions should *not* be recursive: you should just call `foldr` with a proper lambda function.